Lecture 20 · Part V · Extended notes

Information retrieval: dense retrieval

Dataset: SciFact (BEIR), the same 5,183 abstracts

Why these notes exist

Lectures 19–22 sit outside Géron, the course's primary text. For those four lectures *these notes are the primary source*, and they are examined on exactly the same terms as the textbook parts. Everything in the slide deck for this lecture is here, with the arguments written out at length rather than compressed onto a slide; the numbers are the ones the accompanying notebook prints, and every one of them is a measurement on the corpus named above rather than a figure quoted from a paper.

Contents

1 What an index of strings cannot do

Lecture 19 ended with a measurement rather than an opinion: a relevant abstract is missing 40.9% of its claim's words on average, and 28.6% of judged pairs share fewer than half. *Heart attack* and *myocardial infarction*; *lower numbers* and *reduced inoculum*; a mechanism described rather than named.

The requirement

A representation in which two texts that mean the same thing are *close*, whether or not they share a word. Which is precisely what Lectures 17 and 18 built — and the reason Part V comes after Part IV rather than before it.

1.1 The three repairs that do not work

Repair	Why it falls short
Stemming	merges word forms, not concepts — nothing links *infarction* to *attack*
A synonym dictionary	someone must write it, per domain, and it cannot cover paraphrase
Query expansion	adds terms from the top results, so it needs the answer to already be near the top — which is exactly what has failed

Each is a patch on the same assumption: that relevance is visible in shared strings. The assumption is what has to go. This is the shape of every move from feature engineering to learned representations in this course — the patches are not stupid, they are *bounded*, and the bound is the assumption.

1.2 Be fair to the lexical index first

Before replacing BM25, note what it is exactly right about.

- A rare technical term — a gene name, a drug, an identifier — is matched *exactly*, where an embedding may blur it into its neighbours.
- It needs no training data, no accelerator, and no model version to record.
- Its score is explainable: the contribution of every term can be printed, as we did in Lecture 19.
- A new document is indexed in microseconds. No re-encoding, no re-fitting.

Three of those four are operational requirements that outlive whichever model is fashionable, and they are why BM25 is still in the first stage of systems that also run neural rankers.

2 Bi-encoders

Part IV already gave us everything but one idea. Lecture 17 gave a token embedding and a sentence vector from pooling; Lecture 18 gave attention, so that a token's vector depends on its context. Both give a fixed-width vector for a variable-length text.

The one new idea

Put the query and the document in the *same space*, and make the score a dot product. Everything else in this lecture is a consequence of that choice: what can be precomputed, what must be approximated, and what the model is allowed to see.

There is no separate derivation in this lecture for that reason. The mathematics is a normalised inner product, and you have had it since Lecture 8.

2.1 From tokens to one vector

A retrieval score needs one vector per *text*, so the token vectors must be pooled.

Pooling	What it does
`[CLS]`	take the reserved first token's vector — meaningful only if the model was trained to put something there
Mean	average the token vectors, *masking padding*; robust, and what our model uses
Max	component-wise maximum; rarely better, and discards how often a thing was said

Failure condition — masking the padding is not optional

Mean-pooling over the padding tokens makes a text's vector depend on *the batch it was encoded in*. The same abstract gets two different vectors, the retrieval scores change, and nothing raises an error.

2.2 Why the vectors are normalised

Normalise to unit length and the inner product becomes the cosine, which discards magnitude. That is deliberate.

- Magnitude in these encoders tracks length and token frequency, not relevance — so an unnormalised dot product quietly prefers long documents, which is the failure BM25's b was invented for.
- Normalised scores are bounded in $[-1, 1]$, which makes a threshold transferable between corpora.
- And it makes the clustering of Section 6 legitimate: k-means on the sphere is a meaningful thing to do.

The same trap as Lecture 19's length normalisation, arriving from a different direction. Long documents win by default unless something is done about it.

2.3 Encode both sides into one space

$$\text{score}(q, d) \;=\; E(q) \cdot E(d), \qquad E : \text{text} \to \mathbb{R}^{384}.$$

One encoder, applied *separately* to the query and to the document.

Why "separately" is the whole design

$E(d)$ does not depend on the query. So every document can be encoded once, offline, and *stored*; at request time you encode one short query and take 5,183 inner products. The expensive part happens before anyone asks.

Hold on to that sentence. It is also the reason the bi-encoder is the weaker of the two neural models in this lecture.

One encoder or two?

	Shared encoder	**Two encoders**
Parameters	one set	two sets
Suits	symmetric tasks — sentence similarity	asymmetric — short query, long document
Risk	a short query and a long abstract are not the same kind of text	twice the parameters on the same labels

We use a shared encoder because the model we borrow was trained that way. It is a real modelling choice and it should be reported: a paper that says "bi-encoder" without saying which has told you less than it thinks.

2.4 What it costs to store

	Per document	**Whole corpus**
Dense vectors, 384-d float32	1,536 bytes	8.0 MB
Inverted index postings	488 bytes	2.5 MB

Three times the storage — and, unlike postings, *every byte of it is touched by every query*. The inverted index skips the documents sharing no term; the dense index cannot skip anything, because every vector has a non-zero dot product with every query. That is why approximate nearest-neighbour search exists, and it is Section 6.

The dimension is a dial too:

Bytes per document	1,536
This corpus	8.0 MB
The same corpus at 768 dimensions	15.9 MB
Ten million documents at 384-d	15.4 GB

A dense index is expensive not to build but *to use*, and that is what quantisation and approximate search exist to reduce.

2.5 The whole retrieval system, in six lines

```
D = encode(model, documents)              # once, offline, 5,183 x 384
D = D / np.linalg.norm(D, axis=1, keepdims=True)
```

```
def search(query, k=10):
    q = encode(model, [query])[0]
    q /= np.linalg.norm(q)
    s = D @ q                              # every document, exactly
    return np.argsort(-s)[:k]
```

Note what is *not* here: no index structure, no library, no database. The matrix product is the search, and on this corpus it is fast enough to be exact. Everything a vector database adds is a way of not doing that matrix product in full, and it is bought with recall.

3 Training one, and why we will not

3.1 What the objective must say

We want $E(q) \cdot E(d^+)$ large and $E(q) \cdot E(d^-)$ small. Written as a softmax over one positive and a set of negatives:

The contrastive objective

$$\mathcal{L} \;=\; -\log \frac{\exp\big(E(q)\cdot E(d^+)/\tau\big)}{\exp\big(E(q)\cdot E(d^+)/\tau\big) + \sum_{d^-} \exp\big(E(q)\cdot E(d^-)/\tau\big)}$$

which is the cross-entropy of Lecture 17 with the *documents* as classes, and the objective you meet again in Lecture 23 with images on one side.

The positives come from the judgements. The negatives are the problem: there are millions of documents and you cannot score them all.

3.2 In-batch negatives

In a batch of B query–document pairs, every *other* document in the batch is a negative for every query. One encoding pass gives B^2 scores.

Batch B	Texts encoded	Scores available	Negatives per query
8	16	64	7
32	64	1,024	31
128	256	16,384	127
512	1,024	262,144	511

Encoding grows linearly and negatives grow linearly; the *scores* grow quadratically, because the dot products are nearly free next to the encoder. This is why dense-retrieval papers report batch sizes in the thousands, and why the batch size here is a *modelling decision* rather than a memory setting.

Failure condition — "In-batch negatives are free"

The scores are free. The encodings are not, and the score matrix itself is not: at $B = 512$ it is about a megabyte in float32, and at $B = 16{,}384$ it is a gigabyte. What actually caps the batch is memory, and the batch is the number of classes in the problem you are solving.

3.3 The temperature

The τ in the objective is not decoration. Scores are cosines, so they live in $[-1, 1]$; a softmax over that range is nearly uniform and the gradient nearly vanishes. Small τ sharpens the distribution, so the hardest negative dominates the gradient; large τ flattens it, so all negatives contribute almost equally and the model learns slowly.

Typical values are 0.01 to 0.05, which are *small*: dividing a cosine by 0.05 spreads $[-1, 1]$ across $[-20, 20]$, which is where a softmax has gradient to give. Lecture 23 derives this properly.

3.4 Why the negatives are most of the work

A random negative is almost always trivially wrong: a random abstract shares nothing with the claim, so the model learns to separate "about medicine" from "about anything else".

- **Hard negatives** — documents a first stage ranks highly but which are not relevant. These teach the distinction that matters.
- **False negatives** — and here is the trap: a hard negative mined this way is often *relevant but unjudged*, so training actively teaches the model that a correct answer is wrong.

Failure condition — the pooling problem, now inside training

Lecture 19 said unjudged documents are scored as irrelevant *at evaluation*. Here they are scored as irrelevant *during training*, which is worse: the error is no longer an accounting mistake, it is a gradient.

On this corpus, of 1.55 million claim–abstract pairs only 339 are judged relevant, a mean of 1.13 per claim. BM25's top ten for a claim contains on average 0.86 judged-relevant abstracts and 9.14 others — and those others are *unjudged*, not judged irrelevant. Train on them as negatives and the gradient pushes the model away from correct answers. This is the single commonest reason a fine-tuned retriever underperforms the paper it copied.

3.5 Today we do not train one

SciFact has 300 test claims and a training set of a few hundred more. Fine-tuning a bi-encoder on that would measure the split, not the method. So we take a model trained on something else entirely — `all-MiniLM-L6-v2`, trained by contrastive learning on about a billion general sentence pairs scraped from the web, never shown a scientific claim — and apply it **zero shot**.

That is also the honest question. "Should I put an off-the-shelf embedding model in front of my search?" is the decision most of you will actually face. It is not "can a dense retriever be trained to beat BM25 given enough labels?" — the answer to that is yes, and it is not usually the question you have.

Predict before you read on

A model whose training distribution excludes your domain should do worst exactly where the domain vocabulary is most specialised. BM25 scored 0.6611 NDCG@10 with no training at all. Write down a number for the zero-shot dense retriever *now*. The point is not whether you are right; it is to notice, on the next page, whether you would have argued for the answer had you seen it first.

3.6 What the bi-encoder gave up

A single 384-dimensional vector must stand for a 225-token abstract, and it is computed *before the query is known*.

- An abstract about three things compresses to one point, so it is somewhat about each and exactly about none.
- Nothing in the encoder can emphasise the part of the document the query is asking about — it has not been asked yet.
- Exact term matching is not guaranteed: two different gene names may land close together.

That is the price of indexability, and the trade is *structural*: it is not fixed by a bigger encoder. It is the whole reason a second, non-indexable model appears later in this lecture.

3.7 The operational cost the accuracy table never shows

	Inverted index	**Dense index**
New document	append to its terms' postings; microseconds	one encoder pass, then append the vector
New model version	nothing to do	re-encode the entire corpus
Mixed versions	impossible	silently wrong — vectors from two models are not comparable

The last row is a real outage, and a quiet one: nothing errors, the scores are simply meaningless for the documents encoded by the old model. A dense index needs a model version stored beside it.

4 Against BM25, on the same judgements

4.1 The protocol, stated before the numbers

Same corpus of 5,183 abstracts, same 300 test claims, same judgements. Same metric code — the functions written in Lecture 19's notebook, unchanged. Both retrievers scored *exactly*: every document for every query, no approximate index in either. And the metric was named before the run: NDCG@10, with recall@100 reported alongside because that is what a first stage is judged on.

4.2 The result

Metric	BM25	Dense, zero shot
NDCG@10	0.6611	0.6451
Precision@1	0.5433	0.5033
MRR	0.6350	0.6113
Recall@10	0.7784	0.7833
Recall@100	0.8852	0.9250

The neural model loses on the headline metric

Thirty years of age, no training, no accelerator, no vector store — and BM25 is ahead by 0.0160 NDCG@10.

It is not a bug in the experiment: same corpus, same claims, same judgements, same metric code. It is a well-known result — BEIR was built to report it, and out-of-domain dense retrieval loses to BM25 on a good fraction of its tasks. Scientific text is exactly where it should be worst, because the vocabulary is technical and a general-purpose encoder has never seen most of it.

What this result does not license

- It does *not* say dense retrieval is worse in general. Fine-tuned on in-domain pairs it beats BM25 on this corpus, and that is well replicated.
- It does *not* say embeddings are a bad idea. Recall@100 went up, and the mismatch case went from rank 4,999 to rank 8.
- It *does* say that this particular substitution — off-the-shelf encoder for BM25, no labels, technical corpus — is a loss, and that is a decision many teams make without measuring.

Stating the scope of a claim is not hedging. A result whose boundary you cannot draw is a result you cannot use.

4.3 But look at recall@100

Metric	BM25	Dense	Difference
NDCG@10	0.6611	0.6451	−0.0160
Recall@10	0.7784	0.7833	+0.0049
Recall@100	0.8852	0.9250	+0.0398

The dense retriever is *worse at ordering and better at finding*. Recall from Lecture 19 what a first stage is judged on: not what the user sees, but what it makes possible. On that criterion — the criterion for the job it would actually be given — the dense retriever wins.

4.4 Do they find the same documents?

Take every relevant abstract in the test set and ask which method placed it in the top ten.

Found by	Count	Share
Both	227	67.0%
BM25 only	30	8.8%
Dense only	38	11.2%
Neither	44	13.0%

68 of 339 relevant abstracts are found by exactly one of the two. They are not two attempts at the same ranking; they fail on *different documents*.

A mean hides the disagreement

BM25 0.6611 against dense 0.6451 sounds like two systems doing almost the same thing. Per claim they frequently disagree completely. Two systems with the same mean can have entirely different error sets: the mean answers "which is better on average" and cannot answer "are these the same system" — and only the second question tells you whether to combine them.

4.5 The claim BM25 could not find

"In mice, P. chabaudi parasites are able to proliferate faster early in infection when inoculated at lower numbers than when inoculated at high numbers."

Rank of its one relevant abstract	
BM25	4,999
Dense, zero shot	8

Nothing about the text changed; the abstract still shares almost no words with the claim. What changed is that the comparison is no longer between strings. Be precise about what this shows: not that the dense retriever is better — the table above says it is not — but that the two methods fail *independently*.

5 Combining them, and paying for it

5.1 Hybrid retrieval

The scores are not on a comparable scale — BM25 returns unbounded sums, the dense model returns cosines — so fuse the *ranks* instead:

Reciprocal rank fusion

$$\mathrm{RRF}(d) \;=\; \sum_{\text{systems}} \frac{1}{k + \mathrm{rank}(d)}, \qquad k = 60.$$

No training, no tuning, no calibration, and it needs nothing from the two systems but their orderings — which is why it is what people actually deploy. The constant k damps the top ranks so one system cannot dominate on a single confident hit; 60 is conventional, in the same sense that $\log_2(i+1)$ is.

Metric	BM25	Dense	Hybrid
NDCG@10	0.6611	0.6451	0.6948
Precision@1	0.5433	0.5033	0.5633
Recall@10	0.7784	0.7833	0.8239
Recall@100	0.8852	0.9250	0.9617

Better than either, on every metric, from adding two reciprocals. The gain over BM25 is about +0.034 NDCG@10 — larger than anything else in this lecture will buy you. And it is not mysterious: 68 relevant abstracts were found by exactly one system, and fusion is how you keep both.

Why not just add the scores?

The obvious fusion is α BM25 + $(1 - \alpha)$ cosine. It works, and it costs two things. The scales are incomparable and corpus-dependent, so the scores must be normalised and the normalisation is another choice; and α must be tuned on held-out queries, so you now need labels for a method whose selling point was needing none. Rank fusion avoids both. Score fusion can do better when you have the labels to tune it — which is exactly the condition under which you would have fine-tuned the encoder instead.

5.2 Cross-encoders

$$\text{score}(q, d) = \text{Transformer}\big(\,\text{[CLS]}\ q\ \text{[SEP]}\ d\,\big)$$

Concatenate the query and the document and put both through the model *together*. Every token of the query can attend to every token of the document, which is exactly what a bi-encoder forbids by construction.

	Bi-encoder	Cross-encoder
Sees	each text alone	the pair, jointly
Document vectors	precomputed	impossible — they depend on the query
Cost per query	one encoding + N dot products	N transformer passes
Accuracy	lower	higher

One sentence worth being able to say

The accuracy difference and the cost difference have the *same cause*: whether the query is present when the document is read.

Why the accurate model cannot go first

A full cross-encoder scan of this corpus, for these claims, is

$$5{,}183 \times 300 = 1{,}554{,}900 \text{ transformer passes}$$

on the toy corpus of this course. A web index is 10^{10} documents and the user is waiting 200 ms, so the arithmetic is not close — it is off by ten orders of magnitude. Hence the pipeline: a cheap first stage reduces millions to a hundred, and an expensive second stage reorders that hundred. The architecture of every production search system is a direct consequence of the fact that the good model does not scale.

5.3 Re-ranking, measured

Cross-encoder over BM25's top k, everything below k left where it was:

System	NDCG@10	P@1	Pairs scored
BM25 alone	0.6611	0.5433	0
+ re-rank top 10	0.6656	0.5600	3,000
+ re-rank top 50	0.6773	0.5667	15,000
+ re-rank top 100	0.6781	0.5667	30,000

Going from 10 to 50 buys +0.0117; going from 50 to 100 buys +0.0008 for twice the compute. The first ten candidates give most of the gain and the next ninety give a tenth of it. That is the shape of every re-ranking curve you will meet, and the reason production depths are usually in the tens rather than the thousands.

Note also that recall@10 barely moves: re-ranking the top 10 cannot change *which* documents are in the top 10. It changes their order, and P@1 with it.

5.4 The ceiling the re-ranker cannot pass

Re-ranking reorders; it cannot retrieve. So the recall of the first stage at depth k is a hard bound on the second stage at depth k:

Depth k	10	50	100	1000
BM25 recall@k	0.7784	0.8654	0.8852	0.9650

Re-rank BM25's top 100 and a *perfect* cross-encoder still cannot exceed recall 0.8852 at any cutoff — 11.5% of relevant abstracts are simply not in the candidate set. Notice that the hybrid first stage reaches recall@100 of 0.9617.

The most useful sentence in this lecture

Improving the first stage raises the ceiling for everything above it. Improving the second stage never does.

An exam-style question, worked

"A system reports NDCG@10 of 0.6781 using a cross-encoder over the top 100 of a BM25 first stage. Its authors propose replacing the cross-encoder with a larger one. What is the most you can say about the gain, and what would you propose instead?"

- The first stage's recall@100 is 0.8852, so no re-ranker can exceed that recall: 11.5% of relevant documents are absent from the candidates.
- Going from depth 50 to 100 already bought only +0.0008, so the candidates are close to exhausted.
- Propose improving the *first* stage. A hybrid one reaches recall@100 of 0.9617, which raises the ceiling itself.

Section B of the paper is exactly this shape: a table, a proposal, and the question of whether the proposal can possibly work.

6 Approximate nearest neighbours

The dense scan touches every one of the 8.0 MB of vectors for every query, and unlike the inverted index it cannot skip anything. The standard answer, and the one built by hand in the notebook: cluster the vectors once (k-means, 64 clusters), then at query time score only the nearest few clusters.

What this changes about the numbers

Every dense number so far was an exact scan, so it measured the *model*. From here on the numbers measure the model *and the index*, and the two must be reported separately — which is exactly why this whole lecture was built on a corpus small enough to scan exactly.

The claim behind such an index is that a query's nearest documents are mostly in the clusters nearest the query. That is a claim about geometry and it is only approximately true: a document near a cluster boundary is missed when its cluster is not probed, and boundary documents are common in high dimensions. So the index has a *recall*, and that recall is a parameter rather than a property.

Failure condition — index recall and model quality, conflated

Recall of the approximate index is measured against the *exact scan*, never against the relevance judgements. Mixing the two conflates "my index missed it" with "my model ranked it low", and those have different fixes.

Clusters probed	Corpus scanned	Recall@10 vs exact
1	1.8%	0.5047
2	3.6%	0.6580
4	7.3%	0.8013
8	14.1%	0.8920
16	27.9%	0.9577
64	100.0%	1.0000

Probing 4 of 64 scans 7.3% of the corpus and keeps 80.1% of the exact top ten. A dial, not a default — and a vector database ships with it set by somebody who never saw your corpus.

6.1 A latency budget

Suppose 200 ms end to end, with the pipeline as built:

Stage	Model passes	Scales with
BM25	0	query terms × postings length
Encode the query	1	nothing — the query is short
Dense scan (exact)	0	corpus size × 384
Dense scan (probe 4 of 64)	0	7.3% of that
Re-rank top 50	50	the depth you chose

Two of those five rows are dials you set, and both trade quality for time along a curve you can measure. The budget does not tell you where to sit on them; it does tell you that the choice is yours and has to be made deliberately.

7 The whole table, in one place

System	NDCG@10	R@100	Cost per query
Corpus unranked	0.0012	0.0117	nothing
Raw term counting	0.0944	—	postings walk
BM25	0.6611	0.8852	postings walk
Dense, zero shot	0.6451	0.9250	1 encode + full scan
BM25 + cross-encoder@100	0.6781	0.8852	+ 100 transformer passes
Hybrid (BM25 + dense, RRF)	0.6948	0.9617	1 encode + both

The best NDCG@10 in this lecture comes from the row with *no training in it at all* — two untrained retrievers and a sum of reciprocals.

7.1 The arc of the two IR lectures

Step	Because
Boolean retrieval	obvious — and returns nothing for 98% of claims
Score, do not filter	a missing term should cost something, not everything
idf, saturation, length	three named failures of raw counting
Dense retrieval	strings cannot capture paraphrase
Hybrid	the two fail on different documents
Re-ranking	the accurate model does not scale
Approximate index	the dense scan cannot skip anything

Every row is a response to a *measured* failure of the row above.

7.2 How to choose, in practice

Situation	What to do
No labels, technical vocabulary	BM25. Measure before considering anything else
No labels, everyday language, paraphrase matters	hybrid — BM25 plus an off-the-shelf encoder, fused
Thousands of labelled pairs in your domain	fine-tune a bi-encoder; now it can beat BM25
Latency budget allows a second stage	add a cross-encoder over the top 50–100
Corpus too large to scan	approximate index, and report the recall you chose

Every row starts with BM25 in the table. Not because it wins — on your corpus it may not — but because a comparison without it is not a measurement.

Exercise 20.1

Re-rank the *hybrid* top 100 rather than BM25's. The ceiling argument above predicts what should happen to recall; check whether NDCG follows, and say why the two need not move together.

8 Reference

8.1 Five questions for a neural retrieval claim

1. **Is BM25 in the table?** On the same corpus, queries and judgements, tuned no less carefully than the neural model.
2. **Zero-shot or fine-tuned?** And if fine-tuned, on what, and how close is it to the test corpus?
3. **Exact scan or approximate index?** If approximate, at what recall against the exact scan?
4. **Where did the negatives come from?** And what fraction of them are unjudged rather than judged irrelevant?
5. **First stage or whole pipeline?** A re-ranker's NDCG is a property of the candidates it was given.

8.2 Where students lose marks

- Saying a cross-encoder is "a bigger bi-encoder". It is a different computation: joint, and therefore unindexable.
- Claiming a re-ranker improves recall@100. It cannot — it reorders a fixed candidate set.
- Reporting approximate-index recall against the judgements rather than against the exact scan, which conflates two different errors.
- Describing in-batch negatives as "free". The scores are free; the encodings are not, and the batch is the memory limit.
- Concluding that dense retrieval does not work. The claim is narrower: *zero-shot*, on *this* corpus.

8.3 Vocabulary

Bi-encoder	query and document encoded separately; score is a dot product
Cross-encoder	the pair encoded jointly; cannot be precomputed
In-batch negatives	the other documents in the batch, used as negatives
Hard negative	a high-scoring irrelevant document, mined from a first stage
First stage / re-ranker	cheap over everything; expensive over a few
RRF	fusing rankings by summing $1/(k + \text{rank})$
ANN, IVF, nprobe	approximate search; cluster the vectors, probe a few
Zero shot	applied without training on this task or corpus

8.4 Scope

Examinable. Vocabulary mismatch and why lexical repairs are bounded; what a bi-encoder computes and why it is indexable; the contrastive objective and in-batch negatives; why hard negatives matter and how they go wrong; cross-encoders and the first-stage/re-rank argument including the ceiling; hybrid fusion; the recall/latency trade-off of an approximate index.

Not examinable — engineering. Specific model checkpoints, vector-database APIs, HNSW and product-quantisation internals.

Beyond the syllabus. Late interaction (ColBERT), learned sparse retrieval (SPLADE), distillation of cross-encoders into bi-encoders. Each is a direct response to a trade-off this lecture measured, and they are where to read next.

8.5 Reading

- These notes are the primary source.
- Karpukhin et al., *Dense Passage Retrieval* — where in-batch negatives were made to work.
- Thakur et al., *BEIR* — the benchmark built to test zero-shot generalisation, and the source of the result we reproduced.
- Nogueira and Cho, *Passage Re-ranking with BERT* — the cross-encoder as a second stage.
- Johnson et al., *Billion-scale similarity search* — what an IVF index is, done properly.

8.6 Summary

One line to keep

The first stage sets the ceiling; the second stage only decides how close you get to it.

A lexical index matches strings and relevance is about meaning, and no patch on the index closes that gap. A bi-encoder can be indexed because $E(d)$ does not depend on the query — and that same independence is why it is the weaker model. Zero-shot on this corpus it loses to BM25 on NDCG@10, 0.6451 against 0.6611, and wins on recall@100, 0.9250 against 0.8852. The two fail on different documents: 68 of 339 relevant abstracts were found by exactly one, so fusing them beats both at 0.6948. And the accurate model goes second because it does not scale — and it can never exceed the first stage's recall at that depth.

If a lecture in this part has a single practical recommendation, it is this: *before you replace a lexical retriever, try keeping it.*